Psychological Distance and Persuasion in CMV: Mixed-Effects Logistic Regression

Patrick

2026-06-02

Contents

Overview

Mixed-effects logistic regression predicting whether a comment received a delta (`y`) from its construal-level far-proportions (Social and Hypothetical), adjusting for comment-level controls, with a random intercept for thread (post) to account for the within-thread matched design (1 delta : 3 non-delta controls per set). Note that this random-intercept variance was estimated at zero (a singular fit); we retain the mixed model as the a priori specification motivated by the design and discuss this in the convergence section.

Note on the classifier. The Social and Hypothetical labels come from a BERT cascade with modest held-out performance (Social macro-F1 ~0.45, Hypothetical ~0.59). Coefficients should be read with that measurement error in mind; the sensitivity analysis at the end addresses it directly.

```
#install.packages(c("lme4", "tidyverse", "broom.mixed", "performance", "car"))
library(lme4)
library(tidyverse)
library(broom.mixed)   # tidy() for glmer
library(performance)   # collinearity / model checks
```

Load data

```r
# Adjust path if needed:
df <- read_csv("analysis_dataset.csv")

cat("Rows:", nrow(df), "\n")
```

```
## Rows: 81876
```

```r
cat("Deltas (y=1):", sum(df$y == 1), " Non-deltas (y=0):", sum(df$y == 0), "\n")
```

```
## Deltas (y=1): 20469  Non-deltas (y=0): 61407
```

```r
cat("Unique threads:", length(unique(df$thread_id)), "\n")
```

```
## Unique threads: 13740
```

```r
cat("Unique matched sets:", length(unique(df$set_id)), "\n")
```

```
## Unique matched sets: 20469
```

```r
# Types
df <- df %>%
  mutate(
    y          = as.integer(y),
    thread_id  = factor(thread_id),
    set_id     = factor(set_id),
    is_top_level = as.integer(is_top_level)
  )
```

Descriptive comparison

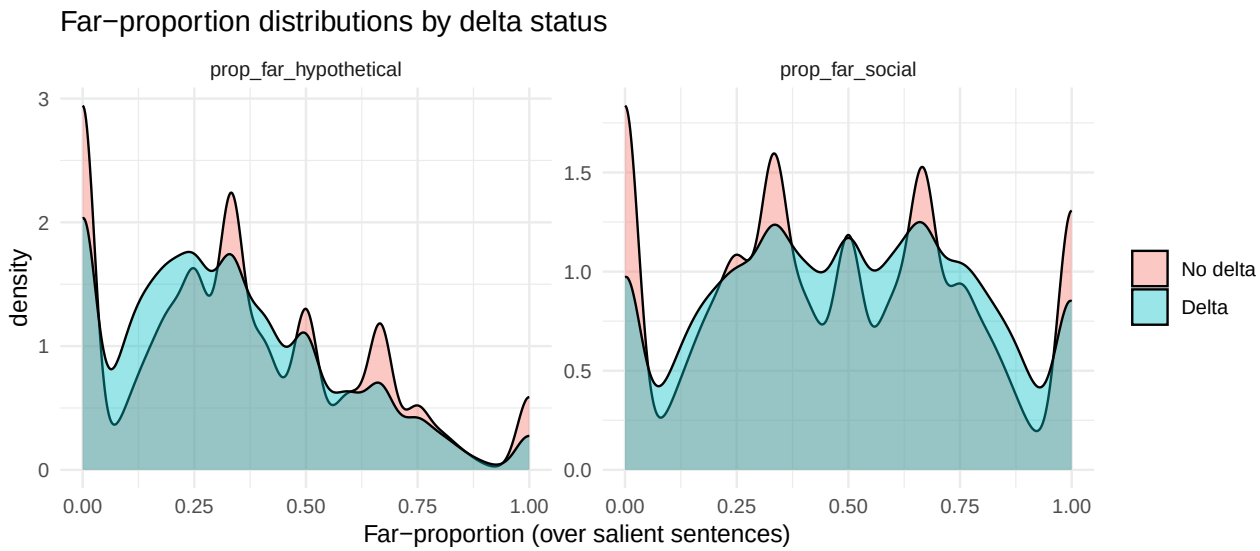
Raw far-proportions by delta status — the unadjusted picture before the model.

```r
df %>%
  group_by(y) %>%
  summarise(
    n = n(),
    far_social_mean = mean(prop_far_social),
    far_social_sd   = sd(prop_far_social),
    far_hypo_mean   = mean(prop_far_hypothetical),
    far_hypo_sd     = sd(prop_far_hypothetical),
    .groups = "drop"
  )
```

```
## # A tibble: 2 x 6
##       y     n far_social_mean far_social_sd far_hypo_mean far_hypo_sd
##   <int> <int>           <dbl>         <dbl>         <dbl>       <dbl>
## 1     0 61407           0.471         0.306         0.347       0.267
## 2     1 20469           0.496         0.288         0.321       0.241
```

```r
df %>%
  pivot_longer(c(prop_far_social, prop_far_hypothetical),
               names_to = "dimension", values_to = "far_prop") %>%
  mutate(delta = factor(y, labels = c("No delta", "Delta"))) %>%
  ggplot(aes(x = far_prop, fill = delta)) +
  geom_density(alpha = 0.4) +
  facet_wrap(~dimension, scales = "free") +
  labs(x = "Far-proportion (over salient sentences)", fill = NULL,
```

```
      title = "Far-proportion distributions by delta status") +
  theme_minimal()
```



Prepare predictors

The continuous controls are highly skewed (author_activity, comment_order, comment_length), so we log-transform then standardise (z-score) them. This helps `glmer` converge and makes coefficients interpretable per standard deviation. The far-proportions are left on their natural 0–1 scale so their odds ratios read as "per unit of proportion" (we also provide a per-0.1 version below).

```
zlog <- function(x) as.numeric(scale(log1p(x)))  # log1p handles zeros (e.g. author_activity = 0)

df <- df %>%
  mutate(
    comment_length_z  = zlog(comment_length),
    comment_order_z   = zlog(comment_order),
    author_activity_z = zlog(author_activity)
  )

summary(df[, c("comment_length_z", "comment_order_z", "author_activity_z")])
```

```
##  comment_length_z  comment_order_z    author_activity_z
##  Min.   :-1.3684   Min.   :-1.86024   Min.   :-2.2755
##  1st Qu.:-0.6867   1st Qu.:-0.73376   1st Qu.:-0.7122
##  Median :-0.2029   Median :-0.03087   Median : 0.0585
##  Mean   : 0.0000   Mean   : 0.00000   Mean   : 0.0000
##  3rd Qu.: 0.6134   3rd Qu.: 0.63282   3rd Qu.: 0.7447
##  Max.   : 4.8235   Max.   : 3.73233   Max.   : 1.8088
```

Primary model

```
m1 <- glmer(
  y ~ prop_far_social + prop_far_hypothetical +
      comment_length_z + is_top_level + comment_order_z + author_activity_z +
      (1 | thread_id),
```

```
  data = df, family = binomial,
  control = glmerControl(optimizer = "bobyqa", optCtrl = list(maxfun = 2e5))
)

summary(m1)
```

```
## Generalized linear mixed model fit by maximum likelihood (Laplace
##   Approximation) [glmerMod]
##  Family: binomial  ( logit )
## Formula: y ~ prop_far_social + prop_far_hypothetical + comment_length_z +
##     is_top_level + comment_order_z + author_activity_z + (1 |      thread_id)
##    Data: df
## Control: glmerControl(optimizer = "bobyqa", optCtrl = list(maxfun = 2e+05))
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##   71271.8   71346.3  -35627.9   71255.8     81868
##
## Scaled residuals:
##     Min      1Q  Median      3Q     Max
## -2.5289 -0.4418 -0.2739  0.0075  7.9543
##
## Random effects:
##  Groups    Name        Variance Std.Dev.
##  thread_id (Intercept) 0        0
## Number of obs: 81876, groups:  thread_id, 13740
##
## Fixed effects:
##                         Estimate Std. Error z value Pr(>|z|)
## (Intercept)           -2.461220   0.028041  -87.77  < 2e-16 ***
## prop_far_social        0.394219   0.032042   12.30  < 2e-16 ***
## prop_far_hypothetical -0.168472   0.038029   -4.43 9.42e-06 ***
## comment_length_z       0.442408   0.009220   47.98  < 2e-16 ***
## is_top_level           1.983625   0.021657   91.59  < 2e-16 ***
## comment_order_z       -0.264584   0.010752  -24.61  < 2e-16 ***
## author_activity_z      0.257879   0.009647   26.73  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##             (Intr) prp_fr_s prp_fr_h cmmnt_l_ is_tp_ cmmnt_r_
## prop_fr_scl -0.651
## prp_fr_hypt -0.551  0.156
## cmmnt_lngt_ -0.082  0.043    0.036
## is_top_levl -0.528  0.039    0.033   -0.085
## cmmnt_rdr_z -0.080 -0.008    0.015   -0.159    0.358
## athr_ctvty_ -0.064  0.010   -0.021    0.054    0.080  0.188
```

Convergence check

```
# Did it converge cleanly? (no warnings printed above = good)
# Singular fit check (random-intercept variance near zero is worth noting):
cat("Random-intercept SD (thread):",
    attr(VarCorr(m1)$thread_id, "stddev"), "\n")
```

```
## Random-intercept SD (thread): 0
```

```
isSingular(m1)
```

```
## [1] TRUE
```

Odds ratios with 95% CIs

```
or_tab <- broom.mixed::tidy(m1, effects = "fixed", conf.int = TRUE,
                            exponentiate = TRUE)
or_tab %>%
  select(term, estimate, conf.low, conf.high, p.value) %>%
  mutate(across(where(is.numeric), ~round(.x, 4)))
```

```
## # A tibble: 7 x 5
##   term                  estimate conf.low conf.high p.value
##   <chr>                    <dbl>    <dbl>     <dbl>   <dbl>
## 1 (Intercept)             0.0853   0.0808    0.0902       0
## 2 prop_far_social         1.48     1.39      1.58         0
## 3 prop_far_hypothetical   0.845    0.784     0.910        0
## 4 comment_length_z        1.56     1.53      1.58         0
## 5 is_top_level            7.27     6.97      7.58         0
## 6 comment_order_z         0.768    0.752     0.784        0
## 7 author_activity_z       1.29     1.27      1.32         0
```

The `estimate` column is the odds ratio. For the far-proportions (0–1 scale), the OR is the multiplicative change in odds of a delta when the proportion goes from 0 to 1 — a large swing. A per-0.1 interpretation is often more meaningful:

```
or_tab %>%
  filter(term %in% c("prop_far_social", "prop_far_hypothetical")) %>%
  mutate(OR_per_0.1 = round(estimate^(0.1), 4)) %>%
  select(term, OR_full_range = estimate, OR_per_0.1, conf.low, conf.high, p.value)
```

```
## # A tibble: 2 x 6
##   term                  OR_full_range OR_per_0.1 conf.low conf.high  p.value
##   <chr>                         <dbl>      <dbl>    <dbl>     <dbl>    <dbl>
## 1 prop_far_social               1.48       1.04     1.39      1.58  8.71e-35
## 2 prop_far_hypothetical         0.845      0.983    0.784     0.910 9.42e- 6
```

Collinearity check

```
performance::check_collinearity(m1)
```

```
## # Check for Multicollinearity
##
## Low Correlation
##
##                   Term  VIF   VIF 95% CI adj. VIF Tolerance Tolerance 95% CI
##        prop_far_social 1.03 [1.02, 1.04]     1.01      0.97     [0.96, 0.98]
##  prop_far_hypothetical 1.03 [1.02, 1.04]     1.01      0.97     [0.97, 0.98]
##       comment_length_z 1.04 [1.03, 1.05]     1.02      0.96     [0.96, 0.97]
##           is_top_level 1.15 [1.14, 1.16]     1.07      0.87     [0.86, 0.87]
##        comment_order_z 1.21 [1.20, 1.22]     1.10      0.83     [0.82, 0.83]
##      author_activity_z 1.05 [1.04, 1.05]     1.02      0.96     [0.95, 0.96]
```

Robustness: nested random effects (optional)

Models comments nested within matched set within thread. Heavier and may be singular (many sets have only 4 comments), so treat as a robustness check, not the primary model.

```
m2 <- glmer(
  y ~ prop_far_social + prop_far_hypothetical +
      comment_length_z + is_top_level + comment_order_z + author_activity_z +
      (1 | thread_id/set_id),
  data = df, family = binomial,
  control = glmerControl(optimizer = "bobyqa", optCtrl = list(maxfun = 2e5))
)
summary(m2)
broom.mixed::tidy(m2, effects = "fixed", conf.int = TRUE, exponentiate = TRUE)
```

Sensitivity analysis: classifier error

Because the Social and Hypothetical labels are classifier predictions rather than gold annotations, the far-proportions carry measurement error, and that error could bias the regression coefficients. We assess robustness with a coarse label-perturbation sweep. For each comment, the far-proportion is treated as a count of "far" labels over its salient sentences; we then randomly flip labels (far→not-far and not-far→far) at a fixed error rate, recompute the far-proportions, and refit the model. We repeat this 30 times at each of four error rates (0%, 15%, 30%, 45%) and report the median odds ratio and 2.5–97.5% range across repetitions. The sweep is deliberately coarse rather than pinned to the test-set confusion matrix. With only ~155 held-out sentences split across three classes, the per-class error rates are themselves too uncertain to support precise per-cell perturbation; sweeping a plausible band is the more honest treatment and keeps the method simple. Because the classifier was trained without access to delta status, its errors are plausibly non-differential with respect to the outcome — meaning they bias coefficients toward the null. The simulation makes this concrete: as the error rate rises, both odds ratios should move monotonically toward 1, so any effect that survives substantial noise is, if anything, an underestimate of the true association. Refitting uses glm rather than glmer, since the thread-level random-intercept variance in the primary model was estimated at zero (see below), making the two estimators equivalent for the fixed effects while glm is far faster across the many refits.

```
set.seed(42)
error_rates <- c(0.0, 0.15, 0.30, 0.45)
n_reps <- 30

# Perturb a proportion by simulating label flips: a far-proportion p over k
# salient sentences ~ k*p far labels; flip each label independently w.p. e.
perturb <- function(p, k, e) {
  k <- pmax(k, 1)
  far  <- round(p * k)
  near <- k - far
  # far->not and not->far flips at rate e
  new_far <- rbinom(length(far), far,  1 - e) + rbinom(length(near), near, e)
  pmin(pmax(new_far / k, 0), 1)
}

sens <- purrr::map_dfr(error_rates, function(e) {
  purrr::map_dfr(seq_len(n_reps), function(r) {
    dfp <- df %>%
      mutate(
        ps = perturb(prop_far_social,       n_salient, e),
```

```
        ph = perturb(prop_far_hypothetical, n_salient, e)
      )
    m <- tryCatch(
      glm(y ~ ps + ph + comment_length_z + is_top_level +
            comment_order_z + author_activity_z,
          data = dfp, family = binomial),   # glm: random effect was ~0 anyway
      error = function(err) NULL)
    if (is.null(m)) return(NULL)
    b <- coef(m)
    tibble(error_rate = e, rep = r,
           OR_social = exp(b["ps"]), OR_hypo = exp(b["ph"]))
  })
})

sens %>%
  group_by(error_rate) %>%
  summarise(
    social_OR_median = median(OR_social),
    social_OR_lo = quantile(OR_social, .025),
    social_OR_hi = quantile(OR_social, .975),
    hypo_OR_median = median(OR_hypo),
    hypo_OR_lo = quantile(OR_hypo, .025),
    hypo_OR_hi = quantile(OR_hypo, .975),
    .groups = "drop"
  )
```

```
## # A tibble: 4 x 7
##   error_rate social_OR_median social_OR_lo social_OR_hi hypo_OR_median
##        <dbl>            <dbl>        <dbl>        <dbl>          <dbl>
## 1       0                1.48         1.48         1.48          0.845
## 2       0.15             1.47         1.41         1.53          0.852
## 3       0.3              1.36         1.28         1.44          0.873
## 4       0.45             1.10         1.03         1.20          0.959
## # i 2 more variables: hypo_OR_lo <dbl>, hypo_OR_hi <dbl>
```

The far-social effect remained significant (OR $>$ 1, CI excluding 1) across all error rates up to 45%, while the far-hypothetical effect held to 30% and attenuated to non-significance only at 45% — consistent with the lower classifier reliability of the Hypothetical Far class. Both effects moved toward the null as noise increased, indicating the primary estimates are conservative.